Министерство образования Республики Беларусь
Учреждение образования
«Брестский государственный технический университет»
Кафедра ИИТ

Лабораторная работа №6
По дисциплине «Современный платформы программирования»
Тема: «освоение приемов тестирования кода на примере использования пакета pytest

Выполнил:
Студент 3 курса
Группы ПО-12
Попов К. В.
Вариант 19

Проверила:
Кулик А. Д.

Брест 2026

Цель:

Цель работы: освоить приемы тестирования кода на примере использования пакета pytest

Задание 1: Написание тестов для мини-библиотеки покупок (shopping.py)

shopping.py

```
"""
Модуль для управления корзиной покупок и логирования операций.
Лабораторная работа №6.
"""
import requests


class Cart:
    """
    Класс, представляющий корзину покупок.
    """
    def __init__(self):
        """Инициализация пустого списка товаров."""
        self.items = []

    def add_item(self, name, price):
        """Добавляет товар в корзину. Выбрасывает ValueError, если цена < 0."""
        if price < 0:
            raise ValueError("Цена не может быть отрицательной")
        self.items.append({"name": name, "price": price})

    def total(self):
        """Возвращает общую стоимость всех товаров в корзине."""
        return sum(item["price"] for item in self.items)

    def apply_discount(self, discount_percent):
        """Применяет скидку в процентах ко всем товарам в корзине."""
        if discount_percent < 0 or discount_percent > 100:
            raise ValueError("Недопустимое значение скидки")

        factor = 1 - (discount_percent / 100)
        for item in self.items:
```

```
            item["price"] *= factor


def log_purchase(item):
    """Отправляет данные о покупке на удаленный сервер."""
    # Добавлен timeout=5, чтобы pylint не ругался на возможное зависание
    requests.post("https://example.com/log", json=item, timeout=5)

# Словарь купонов вынесен на уровень модуля для удобного мокинга
coupons = {"SAVE10": 10, "HALF": 50}

def apply_coupon(cart, coupon_code):
    """Применяет скидку к корзине на основании кода купона."""
    if coupon_code in coupons:
        cart.apply_discount(coupons[coupon_code])
    else:
        raise ValueError("Invalid coupon")
```

test_cart.py

```
"""
Тесты для модуля корзины покупок (shopping.py).
Лабораторная работа №6.
"""
import pytest
import requests
from shopping import Cart, log_purchase, apply_coupon

# pylint: disable=redefined-outer-name
# Это отключит предупреждение о том, что аргумент функции совпадает с именем фикстуры.

@pytest.fixture
def empty_cart():
    """Фикстура для создания пустого экземпляра корзины."""
    return Cart()

def test_add_item(empty_cart):
    """Проверка добавления товара в корзину."""
    empty_cart.add_item("Apple", 10.0)
    assert len(empty_cart.items) == 1
    assert empty_cart.items[0] == {"name": "Apple", "price": 10.0}

def test_negative_price(empty_cart):
```

```
    """Проверка выброса исключения при отрицательной цене товара."""
    with pytest.raises(ValueError):
        empty_cart.add_item("Banana", -5.0)

def test_total(empty_cart):
    """Проверка расчета общей суммы товаров в корзине."""
    empty_cart.add_item("Apple", 10.0)
    empty_cart.add_item("Banana", 20.0)
    assert empty_cart.total() == 30.0

@pytest.mark.parametrize("discount, expected_price", [
    (0, 10.0),    # 0% - цена остаётся прежней
    (50, 5.0),    # 50% - цена уменьшается вдвое
    (100, 0.0)    # 100% - цена становится ноль
])
def test_apply_discount_valid(empty_cart, discount, expected_price):
    """Тестирование корректных значений скидки через параметризацию."""
    empty_cart.add_item("Apple", 10.0)
    empty_cart.apply_discount(discount)
    assert empty_cart.items[0]["price"] == expected_price

@pytest.mark.parametrize("invalid_discount", [
    -10,  # < 0%
    110   # > 100%
])
def test_apply_discount_exceptions(empty_cart, invalid_discount):
    """Проверка выброса исключения при недопустимом проценте скидки."""
    empty_cart.add_item("Apple", 10.0)
    with pytest.raises(ValueError):
        empty_cart.apply_discount(invalid_discount)

def test_log_purchase(monkeypatch):
    """Тестирование логирования покупки с использованием моков."""

    # pylint: disable=too-few-public-methods
    class MockResponse:
        """Вспомогательный класс для имитации ответа сервера."""
        def __init__(self):
            self.called_url = None
            self.called_json = None
```

```
        def mock_post(self, url, json=None, timeout=None):
            """Имитация метода requests.post."""
            _ = timeout  # Используем переменную, чтобы не было ошибки unused argument
            self.called_url = url
            self.called_json = json

    mock_resp = MockResponse()
    monkeypatch.setattr(requests, "post", mock_resp.mock_post)

    test_item = {"name": "Apple", "price": 10.0}
    log_purchase(test_item)

    assert mock_resp.called_url == "https://example.com/log"
    assert mock_resp.called_json == test_item

def test_apply_coupon_valid(empty_cart, monkeypatch):
    """Проверка применения валидного купона с мокингом словаря купонов."""
    empty_cart.add_item("Apple", 100.0)
    monkeypatch.setattr("shopping.coupons", {"MOCK20": 20})
    apply_coupon(empty_cart, "MOCK20")
    assert empty_cart.items[0]["price"] == 80.0

def test_apply_coupon_invalid(empty_cart):
    """Проверка поведения системы при вводе несуществующего купона."""
    with pytest.raises(ValueError, match="Invalid coupon"):
        apply_coupon(empty_cart, "WRONG_COUPON")
```

```
(.venv) PS D:\Pythone\LAB4> pytest test_cart.py -v
========================================================= test session starts =========================================================
platform win32 -- Python 3.14.2, pytest-9.0.3, pluggy-1.6.0 -- d:\Pythone\LAB4\.venv\Scripts\python.exe
cachedir: .pytest_cache
rootdir: D:\Pythone\LAB4
plugins: anyio-4.13.0
collected 11 items

test_cart.py::test_add_item PASSED                                                                                                [  9%]
test_cart.py::test_negative_price PASSED                                                                                          [ 18%]
test_cart.py::test_total PASSED                                                                                                   [ 27%]
test_cart.py::test_apply_discount_valid[0-10.0] PASSED                                                                            [ 36%]
test_cart.py::test_apply_discount_valid[50-5.0] PASSED                                                                            [ 45%]
test_cart.py::test_apply_discount_valid[100-0.0] PASSED                                                                           [ 54%]
test_cart.py::test_apply_discount_exceptions[-10] PASSED                                                                          [ 63%]
test_cart.py::test_apply_discount_exceptions[110] PASSED                                                                          [ 72%]
test_cart.py::test_log_purchase PASSED                                                                                            [ 81%]
test_cart.py::test_apply_coupon_valid PASSED                                                                                      [ 90%]
test_cart.py::test_apply_coupon_invalid PASSED                                                                                    [100%]

========================================================== 11 passed in 0.20s ==========================================================
```

Задание 2

Напишите тесты к реализованным функциям из лабораторной работы №1. Проверьте тривиальные и граничные случае, а также варианты, когда может возникнуть исключительная ситуация. Если при реализации не использовались отдельные функции, необходимо провести рефакторинг кода.

lab1.py

```
"""
Модуль для поиска наибольшего общего префикса (Лабораторная работа №1).
"""


def find_prefix(words):
    """
    Находит самый длинный общий префикс для списка слов.
    Выбрасывает ValueError, если список пуст.
    """
    if not words:
        raise ValueError("Список слов пуст")

    first_word = words[0]
    first_len = len(first_word)
    words_len = len(words)

    for j in range(first_len):
        char = first_word[j]
        for i in range(1, words_len):
            # Если достигли конца одного из слов или символы не совпадают
            if j == len(words[i]) or words[i][j] != char:
                return first_word[:j]
    return first_word

if __name__ == "__main__":
    user_words =[]
    try:
        const = int(input("Введите сколько будет кол-во строк: "))
        for idx in range(const):
            user_word = str(input(f"Введите слово № {idx+1} "))
            user_words.append(user_word)

        result = find_prefix(user_words)
```

```
        print("Самый длинный префикс:", result)
    except ValueError as e:
        print("Ошибка:", e)
```

test_lab1.py

```
"""
Тесты для функции поиска наибольшего общего префикса (Задание 2).
"""
import pytest
from lab1 import find_prefix

def test_find_prefix_common():
    """Тривиальный случай: есть общий префикс."""
    assert find_prefix(["flower", "flow", "flight"]) == "fl"

def test_find_prefix_no_common():
    """Граничный случай: нет общего префикса."""
    assert find_prefix(["dog", "racecar", "car"]) == ""

def test_find_prefix_single_word():
    """Граничный случай: только одно слово в списке."""
    assert find_prefix(["hello"]) == "hello"

def test_find_prefix_identical_words():
    """Тривиальный случай: все слова одинаковые."""
    assert find_prefix(["test", "test", "test"]) == "test"

def test_find_prefix_with_empty_string():
    """Граничный случай: одно из слов пустое."""
    assert find_prefix(["prefix", "pref", ""]) == ""
    assert find_prefix(["", "b", "c"]) == ""

def test_find_prefix_empty_list():
    """Исключительная ситуация: передача пустого списка."""
    with pytest.raises(ValueError, match="Список слов пуст"):
        find_prefix([])
```

```
(.venv) PS D:\Pythone\LAB4> pytest test_lab1.py -v
platform win32 -- Python 3.14.2, pytest-9.0.3, pluggy-1.6.0 -- d:\Pythone\LAB4\.venv\Scripts\python.exe
cachedir: .pytest_cache
rootdir: D:\Pythone\LAB4
plugins: anyio-4.13.0
collected 6 items

test_lab1.py::test_find_prefix_common PASSED                                   [ 16%]
test_lab1.py::test_find_prefix_no_common PASSED                                [ 33%]
test_lab1.py::test_find_prefix_single_word PASSED                              [ 50%]
test_lab1.py::test_find_prefix_identical_words PASSED                          [ 66%]
test_lab1.py::test_find_prefix_with_empty_string PASSED                        [ 83%]
test_lab1.py::test_find_prefix_empty_list PASSED                               [100%]

========================================== 6 passed in 0.07s ==========================================
```

Задание 3

Написать тесты к методу, а затем реализовать сам метод по заданной спецификации.

Варианты:

1) Реализуйте и протестируйте метод **String repeat(String pattern, int repeat)**, который строит строку из указанного паттерна, повторённого заданное количество раз.

Спецификация метода:

repeat ("e", 0) = ""
repeat ("e", 3) = " eee "
repeat (" ABC ", 2) = " ABCABC "
repeat ("e", -2) = ValueError
repeat (None , 1) = TypeError

zadanie3.py

```
"""
Модуль для работы со строками (Задание 3, Вариант 1).
"""


def repeat(pattern, times):
    """
    Строит строку из указанного паттерна, повторённого заданное количество раз.
    """
    if pattern is None:
        raise TypeError("Паттерн не может быть None")
    if times < 0:
        raise ValueError("Количество повторений не может быть отрицательным")
    return pattern * times
```

test_zadan3.py

```python
"""
Тесты для функции repeat (Задание 3, Вариант 1).
"""
import pytest
from zadanie3 import repeat


def test_repeat_zero():
    """Спецификация: repeat("e", 0) = ''"""
    assert repeat("e", 0) == ""


def test_repeat_positive():
    """Спецификация: repeat("e", 3) = 'eee'"""
    assert repeat("e", 3) == "eee"


def test_repeat_multiple_chars():
    """Спецификация: repeat("ABC", 2) = 'ABCABC'"""
    assert repeat("ABC", 2) == "ABCABC"


def test_repeat_negative_times():
    """Спецификация: repeat("e", -2) = ValueError"""
    with pytest.raises(ValueError):
        repeat("e", -2)


def test_repeat_none_pattern():
    """Спецификация: repeat(None, 1) = TypeError"""
    with pytest.raises(TypeError):
        repeat(None, 1)
```

```
(.venv) PS D:\Pythone\LAB4> pytest test_zadan3.py -v
========================================================= test session starts =========================================================
platform win32 -- Python 3.14.2, pytest-9.0.3, pluggy-1.6.0 -- d:\Pythone\LAB4\.venv\Scripts\python.exe
cachedir: .pytest_cache
rootdir: D:\Pythone\LAB4
plugins: anyio-4.13.0
collected 5 items

test_zadan3.py::test_repeat_zero PASSED                                                                                    [ 20%]
test_zadan3.py::test_repeat_positive PASSED                                                                                [ 40%]
test_zadan3.py::test_repeat_multiple_chars PASSED                                                                          [ 60%]
test_zadan3.py::test_repeat_negative_times PASSED                                                                          [ 80%]
test_zadan3.py::test_repeat_none_pattern PASSED                                                                            [100%]

========================================================== 5 passed in 0.06s ==========================================================
```

Вывод: я освоил приемы тестирования кода на примере использования пакета pytest